

HTTP, REST API এবং Fetch API — Frontend থেকে Backend-এর সাথে সংযোগ

Chapter 21-এ আমরা Asynchronous JavaScript ও Promise শিখেছি। একজন Full Stack ইঞ্জিনিয়ার হিসেবে ক্লায়েন্ট ব্রাউজার কীভাবে সার্ভারের সাথে যোগাযোগ করে, ডাটা আনা-নেওয়া করে এবং ব্যাকএন্ড ডাটাবেজে প্রভাব ফেলে—সেটি গভীরভাবে জানা অপরিহার্য।

Browser Client → HTTP Request (Method, URL, Headers, Body) → Backend REST API → HTTP Response (Status, JSON)

React, Next.js, Node.js বা Express—যেকোনো ওয়েব প্ল্যাটফর্মে ফুল-স্ট্যাক ডেভলপমেন্টের মূল মেরুদণ্ড হলো এই নেটওয়ার্কিং জ্ঞান। 🔥

Part 01 — Networking Protocols & HTTP Client-Server Architecture (22.1 – 22.12)

22.1 API কী?

API = Application Programming Interface

সহজভাবে:

API হলো এমন একটি মাধ্যম যার মাধ্যমে একটি application অন্য application/service-এর সাথে কথা বলে এবং data আদান-প্রদান করে।

💡 Real-Life Example 🍔

ধরো তুমি restaurant-এ গেলে।

```
You
↓
Waiter
↓
Kitchen
↓
Food
↓
Waiter
↓
You
```

এখানে **Waiter = API**

তুমি সরাসরি kitchen-এ গিয়ে order দিচ্ছ না।

একইভাবে:

```
Frontend
↓
API
↓
Backend
↓
Database
```

Backend-এর database-এ frontend সরাসরি access না করে API-এর মাধ্যমে request করে।

22.2 Frontend এবং Backend Communication

ধরো একটি e-commerce website:

```
React Frontend
  ↓
GET /api/products
  ↓
Node.js / Express Backend
  ↓
MongoDB
  ↓
Backend
  ↓
JSON Response
  ↓
React
```

Frontend:

"আমাকে সব products দাও।"

Backend:

"এই নাও products-এর data।"

22.3 HTTP কী?

HTTP = HyperText Transfer Protocol

এটি web-এর communication protocol।

Browser এবং server HTTP-এর মাধ্যমে communicate করতে পারে।

```
Client
  ↓ HTTP Request
Server
  ↓ HTTP Response
Client
```

Client বলতে কী?

সাধারণত:

- Browser
- Mobile App
- React App
- Postman
- অন্য কোনো application

Server বলতে কী?

যে system request receive করে এবং response দেয়।

যেমন:

- Node.js server
- Express server
- Django
- Spring Boot
- Laravel

22.4 HTTP Request

একটি request-এ সাধারণত থাকে:

```
HTTP Request
├── Method
├── URL
├── Headers
├── Body
└── অন্যান্য তথ্য
```

Example:

```
22.4 HTTP PROTOCOL
GET /api/products
```

HTTP/1.1

22.5 HTTP Response

Server সাধারণত দেয়:

```
HTTP Response
├── Status Code
├── Headers
└── Body
```

Example:

22.5 HTTP PROTOCOL

HTTP/1.1

```
HTTP/1.1 200 OK
Content-Type: application/json
```

Body:

22.5 JSON PAYLOAD

APPLICATION/JSON

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 70000
  }
]
```

22.6 HTTP Methods

সবচেয়ে গুরুত্বপূর্ণ HTTP methods:

Method	কাজ
GET	Data নেওয়া
POST	নতুন Data তৈরি করা
PUT	সম্পূর্ণ resource update করা
PATCH	Resource-এর নির্দিষ্ট অংশ update করা
DELETE	Data delete করা
HEAD	Header information নেওয়া
OPTIONS	Server কী methods/options support করে তা জানা

Frontend development-এ সবচেয়ে বেশি:

OUTPUT / TEXT

```
GET
POST
PUT
PATCH
DELETE
```

22.7 GET

GET ব্যবহার করা হয় data নেওয়ার জন্য।

Example:

22.7 HTTP PROTOCOL

HTTP/1.1

```
GET /api/products
```

JavaScript:

22.7 JAVASCRIPT FETCH API

ES2026

```
const response = await fetch("/api/products");
const products = await response.json();
console.log(products);
```

22.8 POST

POST সাধারণত নতুন resource তৈরি করতে ব্যবহার করা হয়।

Example:

22.8 HTTP PROTOCOL

HTTP/1.1

POST /api/products

Data:

22.8 JSON PAYLOAD

APPLICATION/JSON

```
{  
  "name": "Laptop",  
  "price": 70000  
}
```

JavaScript:

22.8 JAVASCRIPT FETCH API

ES2026

```
const response = await fetch("/api/products", {  
  method: "POST",  
  
  headers: {  
    "Content-Type": "application/json"  
  },  
  
  body: JSON.stringify({  
    name: "Laptop",  
    price: 70000  
  })  
});  
  
const data = await response.json();  
  
console.log(data);
```

22.9 PUT

PUT সাধারণত একটি resource-এর সম্পূর্ণ representation replace/update করতে ব্যবহার করা হয়।

22.9 JAVASCRIPT FETCH API

ES2026

```
await fetch("/api/products/1", {  
  method: "PUT",  
  
  headers: {  
    "Content-Type": "application/json"  
  },  
  
  body: JSON.stringify({  
    name: "Gaming Laptop",  
    price: 90000  
  })  
});
```

22.10 PATCH

PATCH সাধারণত resource-এর নির্দিষ্ট অংশ update করতে ব্যবহার করা হয়।

ধরা শুধু price পরিবর্তন করবে:

22.10 JAVASCRIPT FETCH API

ES2026

```
await fetch("/api/products/1", {  
  method: "PATCH",  
  
  headers: {  
    "Content-Type": "application/json"  
  },  
  
  body: JSON.stringify({  
    price: 85000  
  })  
});
```

22.11 DELETE

Data delete করতে:

22.11 JAVASCRIPT FETCH API

ES2026

```
await fetch("/api/products/1", {
  method: "DELETE"
});
```

22.12 CRUD

Backend/API development-এ একটি খুব গুরুত্বপূর্ণ term:

CRUD

Letter	Meaning	HTTP
C	Create	POST
R	Read	GET
U	Update	PUT/PATCH
D	Delete	DELETE

Real-life product system:

```
Create Product → POST
Get Products → GET
Update Product → PUT/PATCH
Delete Product → DELETE
```

Part 02 — REST API Architecture, Resource URLs & Status Codes (22.13 – 22.25)

22.13 REST API কী?

REST = Representational State Transfer

REST হলো web API design করার একটি architectural style।

REST API সাধারণত resource-oriented হয়।

Example:

OUTPUT / TEXT

```
/api/users
/api/products
/api/orders
/api/students
```

22.14 RESTful URL Design

❌ সাধারণত এভাবে না:

OUTPUT / TEXT

```
/getUsers
/createProduct
/deleteProduct
/updateProduct
```

✅ Resource-oriented:

OUTPUT / TEXT

```
GET /users
POST /users
GET /users/10
PATCH /users/10
DELETE /users/10
```

Method বলে দেয় কী action হবে।

22.15 REST API Example

ধরো Student API:

OUTPUT / TEXT

```
GET /api/students
GET /api/students/101
POST /api/students
PATCH /api/students/101
DELETE /api/students/101
```

এখানে:

OUTPUT / TEXT

```
/students
```

হলো resource।

22.16 URL কী?

URL:

Uniform Resource Locator

Example:

OUTPUT / TEXT

```
https://example.com/api/products/10
```

এখানে:

OUTPUT / TEXT

```
https://
```

→ Protocol

OUTPUT / TEXT

```
example.com
```

→ Domain

OUTPUT / TEXT

```
/api/products/10
```

→ Path

22.17 Query Parameters

ধরো products search করতে চাই:

OUTPUT / TEXT

```
/api/products?search=laptop
```

এখানে:

OUTPUT / TEXT

```
search=laptop
```

হলো query parameter।

আরেকটি:

OUTPUT / TEXT

```
/api/products?page=2&limit=10
```

এখানে:

OUTPUT / TEXT

```
page=2
limit=10
```

22.18 JavaScript দিয়ে Query Parameters

22.18 JAVASCRIPT FETCH API

ES2026

```
const params = new URLSearchParams({
  search: "laptop",
  page: 2,
  limit: 10
});

const url = `/api/products?${params}`;

console.log(url);
```

CONSOLE OUTPUT

/api/products?search=laptop&page=2&limit=10

22.19 URLSearchParams

এটি query parameter তৈরি/পরিচালনা করতে সাহায্য করে।

22.19 JAVASCRIPT FETCH API

ES2026

```
const params = new URLSearchParams();

params.set("search", "phone");
params.set("page", "1");

console.log(params.toString());
```

OUTPUT / TEXT

search=phone&page=1

22.20 Path Parameter বনাম Query Parameter

⚙️ Path Parameter

OUTPUT / TEXT

/api/products/10

এখানে:

OUTPUT / TEXT

10

নির্দিষ্ট product identify করছে।

⚙️ Query Parameter

OUTPUT / TEXT

/api/products?category=phone

এখানে:

OUTPUT / TEXT

category=phone

filter/search-এর মতো optional information।

সহজভাবে:

OUTPUT / TEXT

/products/10

↑
নির্দিষ্ট ID

/products?category=phone

↑
filter/query

22.21 HTTP Status Code

Server response-এর একটি গুরুত্বপূর্ণ অংশ হলো status code।

1xx — Informational

OUTPUT / TEXT

100 Continue

2xx — Success

OUTPUT / TEXT

200 OK
201 Created
202 Accepted
204 No Content

3xx — Redirection

OUTPUT / TEXT

301 Moved Permanently
302 Found
304 Not Modified

4xx — Client Error

OUTPUT / TEXT

400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Unprocessable Content
429 Too Many Requests

5xx — Server Error

OUTPUT / TEXT

500 Internal Server Error
502 Bad Gateway
503 Service Unavailable
504 Gateway Timeout

22.22 সবচেয়ে গুরুত্বপূর্ণ Status Codes

Code	Meaning
200	Request successful
201	Resource created
204	Success, no response body
400	Bad request
401	Authentication required/failed
403	Access forbidden
404	Resource not found
409	Conflict
422	Validation/unprocessable content
429	Too many requests
500	Server error
503	Service unavailable

22.23 response.ok

Fetch-এর response object-এ:

22.23 JAVASCRIPT FETCH API

ES2026

`response.ok`

থাকে।

সাধারণভাবে 200–299 status হলে এটি `true`।

Example:

22.23 JAVASCRIPT FETCH API

ES2026

```
const response = await fetch("/api/users");  
  
console.log(response.ok);
```

22.24 response.status

22.24 JAVASCRIPT FETCH API

ES2026

```
const response = await fetch("/api/users");  
  
console.log(response.status);
```

Possible output:

OUTPUT / TEXT

200

22.25 Complete Fetch Error Handling

22.25 JAVASCRIPT FETCH API

ES2026

```
async function getUsers() {  
  try {  
    const response = await fetch("/api/users");  
  
    if (!response.ok) {  
      throw new Error(  
        `HTTP Error: ${response.status}`  
      );  
    }  
  
    const users = await response.json();  
  
    console.log(users);  
  } catch (error) {  
    console.error("Request failed:", error);  
  }  
}
```

এটি একটি খুব useful pattern।

22.26 Response Headers

Response-এর headers দেখতে:

22.26 JAVASCRIPT FETCH API

ES2026

```
const response = await fetch("/api/users");

console.log(response.headers);
```

নিদিষ্ট header:

22.26 JAVASCRIPT FETCH API

ES2026

```
console.log(
  response.headers.get("Content-Type")
);
```

22.27 Request Headers

Server-কে information পাঠাতে headers ব্যবহার করা হয়।

22.27 JAVASCRIPT FETCH API

ES2026

```
fetch("/api/users", {
  headers: {
    "Content-Type": "application/json"
  }
});
```

22.28 Authorization Header

Authentication-এর ক্ষেত্রে অনেক API-তে:

22.28 JAVASCRIPT FETCH API

ES2026

```
fetch("/api/profile", {
  headers: {
    "Authorization": "Bearer YOUR_TOKEN"
  }
});
```

ব্যবহার করা হয়।

এখানে:

OUTPUT / TEXT

Authorization

হলো header।

Bearer YOUR_TOKEN

হলো header value।

Real application-এ token handling নিরাপদভাবে করতে হয়; token অযথা expose করা উচিত নয়।

22.29 Request Body

POST request:

22.29 JAVASCRIPT FETCH API

ES2026

```
const user = {
  name: "Shariar",
  email: "shariar@example.com"
};

const response = await fetch("/api/users", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(user)
});
```

এখানে:

22.29 JAVASCRIPT FETCH API

ES2026

```
JSON.stringify(user)
```

object-কে JSON string-এ convert করছে।

22.30 কেন JSON.stringify() ?

JavaScript object:

22.30 JAVASCRIPT FETCH API

ES2026

```
{
  name: "Shariar",
  age: 24
}
```

JSON:

22.30 JSON PAYLOAD

APPLICATION/JSON

```
{
  "name": "Shariar",
  "age": 24
}
```

HTTP request body-তে JSON পাঠাতে সাধারণত:

22.30 JAVASCRIPT FETCH API

ES2026

```
JSON.stringify()
```

ব্যবহার করা হয়।

22.31 Response থেকে JSON

22.31 JAVASCRIPT FETCH API

ES2026

```
const response = await fetch("/api/users");
const data = await response.json();
```

`response.json()` একটি Promise return করে।

তাই:

22.31 JAVASCRIPT FETCH API

ES2026

```
await response.json()
```

ব্যবহার করা হয়।

22.32 Response Object

Fetch করলে আমরা একটি `Response` object পাই।

22.32 JAVASCRIPT FETCH API

ES2026

```
const response = await fetch(url);
```

এর গুরুত্বপূর্ণ properties/methods:

OUTPUT / TEXT

```
response.ok  
response.status  
response.statusText  
response.headers  
response.url  
response.json()  
response.text()  
response.blob()  
response.arrayBuffer()  
response.formData()
```

22.33 response.text()

যদি server plain text দেয়:

22.33 JAVASCRIPT FETCH API

ES2026

```
const response = await fetch("/message");  
  
const text = await response.text();  
  
console.log(text);
```

22.34 response.json()

JSON response:

22.34 JAVASCRIPT FETCH API

ES2026

```
const data = await response.json();
```

সবচেয়ে common।

22.35 response.blob()

Image/file-এর মতো binary data handle করতে `blob()` ব্যবহার করা যায়।

22.35 JAVASCRIPT FETCH API

ES2026

```
const response = await fetch("/image.jpg");  
  
const blob = await response.blob();  
  
console.log(blob);
```

22.36 AbortController

কখনো request cancel করতে হয়।

যেমন:

- User search করছে
- পুরোনো search request cancel করা দরকার
- Page change হয়েছে
- Request timeout করতে হবে

তখন:

22.36 JAVASCRIPT FETCH API

ES2026

```
const controller = new AbortController();

fetch("/api/data", {
  signal: controller.signal
});
```

Cancel:

22.36 JAVASCRIPT FETCH API

ES2026

```
controller.abort();
```

22.37 Abort Example

22.37 JAVASCRIPT FETCH API

ES2026

```
const controller = new AbortController();

async function loadData() {
  try {
    const response = await fetch("/api/data", {
      signal: controller.signal
    });
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log(error.name);
  }
}

loadData();

// Later
controller.abort();
```

Abort হলে সাধারণত error-এর name:

OUTPUT / TEXT

AbortError

হতে পারে।

22.38 Fetch Timeout

Fetch-এর জন্য নিজে timeout implement করা যায় `AbortController` দিয়ে।

22.38 JAVASCRIPT FETCH API

ES2026

```
async function fetchWithTimeout(url, ms) {  
  const controller = new AbortController();  
  
  const timer = setTimeout(() => {  
    controller.abort();  
  }, ms);  
  
  try {  
    const response = await fetch(url, {  
      signal: controller.signal  
    });  
  
    return response;  
  } finally {  
    clearTimeout(timer);  
  }  
}
```

ব্যবহার:

22.38 JAVASCRIPT FETCH API

ES2026

```
try {  
  const response = await fetchWithTimeout(  
    "/api/data",  
    5000  
  );  
  
  console.log(await response.json());  
} catch (error) {  
  console.log("Request failed or timed out");  
}
```

Part 04 — Web Security: CORS, Preflights, Auth & UI States (22.39 – 22.50)

22.39 CORS

🔥 CORS খুব গুরুত্বপূর্ণ

CORS = Cross-Origin Resource Sharing

ধরো:

Frontend:

OUTPUT / TEXT

http://localhost:3000

Backend:

OUTPUT / TEXT

http://localhost:5000

দুইটি different origin।

Browser security-এর কারণে server-কে cross-origin request allow করতে হতে পারে।

22.40 Origin কী?

Origin-এর তিনটি প্রধান অংশ:

OUTPUT / TEXT

scheme + host + port

Example:

OUTPUT / TEXT

https://example.com:443

এগুলো different origin হতে পারে:

OUTPUT / TEXT

http://example.com
https://example.com

কারণ scheme আলাদা।

OUTPUT / TEXT

https://example.com
https://api.example.com

কারণ host আলাদা।

OUTPUT / TEXT

https://example.com:443
https://example.com:8080

কারণ port আলাদা।

22.41 CORS Error Example

Frontend:

22.41 JAVASCRIPT FETCH API

ES2026

```
fetch("http://localhost:5000/api/users");
```

Backend CORS allow না করলে browser console-এ CORS-related error দেখা যেতে পারে।

এটা সাধারণত frontend JavaScript-এর syntax error নয়।

Server-এর CORS configuration-এর সাথে সম্পর্কিত।

22.42 Preflight Request

কিছু cross-origin request-এর আগে browser:

22.42 HTTP PROTOCOL

HTTP/1.1

OPTIONS

request পাঠাতে পারে।

এটাকে বলা হয়:

Preflight Request

এটি server-কে জিজ্ঞেস করে:

"এই ধরনের cross-origin request আমি করতে পারব?"

Server appropriate CORS headers দিয়ে অনুমতি দিতে পারে।

22.43 CORS Headers

Server response-এ এমন header থাকতে পারে:

22.43 HTTP PROTOCOL

HTTP/1.1

Access-Control-Allow-Origin: <https://example.com>

এটি নির্দিষ্ট origin allow করতে পারে।

অরও headers:

OUTPUT / TEXT

Access-Control-Allow-Methods

Access-Control-Allow-Headers

Access-Control-Allow-Credentials

22.44 Frontend থেকে Backend Example

ধরো:

Frontend
<http://localhost:5173>

Backend
<http://localhost:5000>

Frontend:

22.44 JAVASCRIPT FETCH API

ES2026

```
const response = await fetch(  
  "http://localhost:5000/api/products"  
);  
  
const products = await response.json();  
  
console.log(products);
```

Backend-কে CORS appropriately configure করতে হবে।

22.45 API Data Rendering

ধরো API থেকে:

22.45 JSON PAYLOAD

APPLICATION/JSON

```
[
  {
    "id": 1,
    "name": "Laptop",
    "price": 70000
  },
  {
    "id": 2,
    "name": "Phone",
    "price": 30000
  }
]
```

JavaScript:

22.45 JAVASCRIPT FETCH API

ES2026

```
async function loadProducts() {

  const response = await fetch("/api/products");

  const products = await response.json();

  const list = document.querySelector("#productList");

  products.forEach(product => {

    const li = document.createElement("li");

    li.textContent =
      `${product.name} - ₳${product.price}`;

    list.appendChild(li);

  });

  loadProducts();
}
```

HTML:

22.45 HTML

MARKUP

```
<ul id="productList"></ul>
```

22.46 Loading, Success, Error State

Modern frontend application-এ API call-এর তিনটি গুরুত্বপূর্ণ UI state:

OUTPUT / TEXT

Loading
Success
Error

Example:

22.46 JAVASCRIPT FETCH API

ES2026

```
async function loadProducts() {  
  
    const status = document.querySelector("#status");  
  
    try {  
  
        status.textContent = "Loading ...";  
  
        const response = await fetch("/api/products");  
  
        if (!response.ok) {  
            throw new Error("Failed to load");  
        }  
  
        const products = await response.json();  
  
        status.textContent =  
            `${products.length} products loaded`;  
  
    } catch (error) {  
  
        status.textContent =  
            "Could not load products.";  
  
    }  
  
}
```

22.47 Search API Example

ধরো user search box-এ লিখল:

OUTPUT / TEXT

laptop

Frontend:

22.47 JAVASCRIPT FETCH API

ES2026

```
async function searchProducts(query) {  
  
    const params = new URLSearchParams({  
        search: query  
    });  
  
    const response = await fetch(  
        `/api/products?${params}`  
    );  
  
    if (!response.ok) {  
        throw new Error("Search failed");  
    }  
  
    return response.json();  
}
```

Call:

22.47 JAVASCRIPT FETCH API

ES2026

```
const products =  
    await searchProducts("laptop");  
  
console.log(products);
```

22.48 POST Form Data

ধরো signup form:

22.48 JAVASCRIPT FETCH API

ES2026

```
const form = document.querySelector("#signupForm");

form.addEventListener("submit", async (event) => {

  event.preventDefault();

  const formData = new FormData(form);

  const user = Object.fromEntries(formData);

  const response = await fetch("/api/register", {

    method: "POST",

    headers: {

      "Content-Type": "application/json"

    },

    body: JSON.stringify(user)

  });

  const result = await response.json();

  console.log(result);

});
```

এখানে Chapter 17-এর **FormData** + Chapter 19-এর **JSON** + Chapter 21-এর **async/await** + Chapter 22-এর **HTTP/API** একসাথে ব্যবহার হয়েছে।

22.49 API Authentication

একটি private API:

OUTPUT / TEXT

GET /api/profile

request-এর সাথে authentication information লাগতে পারে।

Example:

22.49 JAVASCRIPT FETCH API

ES2026

```
const response = await fetch("/api/profile", {

  headers: {

    Authorization: `Bearer ${token}`

  }

});
```

Flow:

```
Login
↓
Server verifies credentials
↓
Authentication result/token
↓
Client sends credentials/token
↓
Protected API
```

Authentication এবং authorization এক জিনিস নয়।

Authentication

তুমি কে?

Authorization

তুমি কী করতে পারবে?

22.50 401 vs 403

401 Unauthorized

সাধারণত authentication প্রয়োজন বা credentials invalid/missing।

OUTPUT / TEXT

"তুমি কে?"

403 Forbidden

Server request বুঝেছে, কিন্তু access অনুমোদিত নয়।

OUTPUT / TEXT

"তুমি কে জানি, কিন্তু এই কাজের permission নেই।"

Part 05 — Enterprise Patterns: Pagination, DevTools & Full CRUD (22.51 – 22.69)

22.51 API Pagination

ধরো database-এ:

OUTPUT / TEXT

100,000 products

একবারে সব পাঠানো inefficient।

তাই pagination:

OUTPUT / TEXT

/api/products?page=1&limit=20

Page 1:

OUTPUT / TEXT

Products 1-20

Page 2:

OUTPUT / TEXT

Products 21-40

22.52 Pagination Example

22.52 JAVASCRIPT FETCH API

ES2026

```
async function getProducts(page = 1) {  
  const params = new URLSearchParams({  
    page,  
    limit: 20  
  });  
  
  const response = await fetch(  
    `/api/products?${params}`  
  );  
  
  return response.json();  
}
```

22.53 API Filtering

OUTPUT / TEXT

/api/products?category=laptop

JavaScript:

22.53 JAVASCRIPT FETCH API

ES2026

```
const params = new URLSearchParams({  
  category: "laptop"  
});  
  
const response = await fetch(  
  `/api/products?${params}`  
);
```

22.54 API Sorting

OUTPUT / TEXT

```
/api/products?sort=price&order=asc
```

অথবা:

OUTPUT / TEXT

```
/api/products?sort=createdAt&order=desc
```

Backend এই parameters অনুযায়ী data sort করতে পারে।

22.55 API Request-এর Full Structure

একটি request:

Method GET / POST / PATCH / DELETE
URL
Headers
Body

Response:

Status Code
Headers
Body

22.56 GET vs POST

বিষয়	GET	POST
Purpose	Data নেওয়া	Data তৈরি/submit
Body	সাধারণত ব্যবহার করা হয় না	সাধারণত থাকে
Query params	Common	Common
Data modification	সাধারণত নয়	হতে পারে
Idempotent	সাধারণত হ্যাঁ	সাধারণত নয়
Cache	Cacheable হতে পারে	সাধারণত defaultভাবে নয়

22.57 PUT vs PATCH

PUT	PATCH
সম্পূর্ণ resource replace/update-এর জন্য	আংশিক update-এর জন্য
পুরো representation পাঠানোর pattern common	শুধু পরিবর্তিত field পাঠানো common

Example:

PUT:

22.57 JSON PAYLOAD

APPLICATION/JSON

```
{
  "name": "Shariar",
  "email": "new@example.com",
  "age": 24
}
```

PATCH:

22.57 JSON PAYLOAD

APPLICATION/JSON

```
{
  "email": "new@example.com"
}
```

22.58 Idempotency

এটি একটু advanced কিন্তু গুরুত্বপূর্ণ।

কোনো operation একইভাবে বারবার করলে final state একই থাকে কি না—এটি idempotency-এর ধারণার সাথে সম্পর্কিত।

উদাহরণ:

OUTPUT / TEXT

PUT /users/1

একই update বারবার পাঠালে সাধারণত একই final state পাওয়া যায়।

অন্যদিকে:

OUTPUT / TEXT

POST /orders

একই request বারবার পাঠালে multiple order তৈরি হতে পারে, যদি server-side ব্যবস্থা না থাকে।

22.59 HTTP Request—এর Real-Life Example

তুমি e-commerce website-এ:

Product দেখতে:

22.59 HTTP PROTOCOL

HTTP/1.1

GET /api/products/10

Cart-এ add:

22.59 HTTP PROTOCOL

HTTP/1.1

POST /api/cart

Quantity update:

22.59 HTTP PROTOCOL

HTTP/1.1

PATCH /api/cart/10

Cart item remove:

22.59 HTTP PROTOCOL

HTTP/1.1

DELETE /api/cart/10

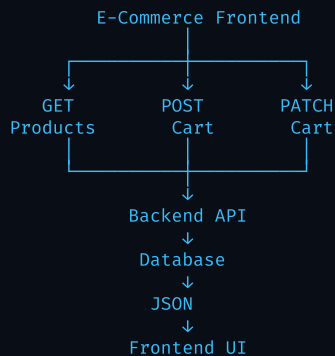
Order create:

22.59 HTTP PROTOCOL

HTTP/1.1

POST /api/orders

22.60 Complete E-Commerce API Flow



22.61 API Error Handling — Professional Pattern

22.61 JAVASCRIPT FETCH API

ES2026

```
async function apiRequest(url, options = {}) {
  try {
    const response = await fetch(url, options);

    if (!response.ok) {
      throw new Error(
        `Request failed: ${response.status}`
      );
    }

    const contentType =
      response.headers.get("content-type");

    if (contentType?.includes("application/json")) {
      return await response.json();
    }

    return await response.text();
  } catch (error) {
    console.error("API Error:", error);
    throw error;
  }
}
```

ব্যবহার:

22.61 JAVASCRIPT FETCH API

ES2026

```
const users = await apiRequest("/api/users");
console.log(users);
```

এভাবে একটি reusable API utility তৈরি করা যায়।

22.62 fetch() কি automatically JSON বুঝে?

❌ না।

এই:

22.62 JAVASCRIPT FETCH API

ES2026

```
const response = await fetch(url);
```

শুধু response দেয়।

JSON পেতে:

22.62 JAVASCRIPT FETCH API

ES2026

```
const data = await response.json();
```

করতে হবে।

22.63 fetch() কি 404-এ automatically reject করে?

❌ সাধারণ HTTP 4xx/5xx status-এর জন্য সাধারণত না।

তাই:

22.63 JAVASCRIPT FETCH API

ES2026

```
if (!response.ok) {
  throw new Error("Request failed");
}
```

check করা গুরুত্বপূর্ণ।

Network-level failure হলে Fetch Promise reject করতে পারে।

22.64 Network Error

যেমন:

OUTPUT / TEXT

```
Internet disconnected
DNS problem
Connection failed
Request aborted
```

এ ধরনের ক্ষেত্রে `fetch()` reject করতে পারে।

তাই:

22.64 JAVASCRIPT FETCH API

ES2026

```
try {
  ...
} catch (error) {
  ...
}
```

গুরুত্বপূর্ণ।

22.65 Browser DevTools — Network Tab

API debugging-এর জন্য:

```
Chrome DevTools
↓
Network
```

এখানে দেখতে পারবে:

- Request URL
- Request Method
- Status Code
- Request Headers
- Response Headers
- Request Payload
- Response
- Timing
- Initiator

Frontend developer হিসেবে **Network tab** ভালোভাবে জানা খুব জরুরি।

22.66 Network Debugging Example

ধরো API কাজ করছে না।

Network tab-এ দেখলে:

OUTPUT / TEXT

```
Request:
POST /api/login
```

```
Status:
401
```

তাহলে authentication-related problem হতে পারে।

অথবা:

OUTPUT / TEXT

```
404
```

হলে endpoint ভুল হতে পারে।

অথবা:

OUTPUT / TEXT

```
500
```

হলে server-side error হতে পারে।

22.67 REST API-এর একটি সম্পূর্ণ Example

ধরো Student Management System।

Get all students

22.67 JAVASCRIPT FETCH API

ES2026

```
async function getStudents() {  
  
    const response =  
        await fetch("/api/students");  
  
    if (!response.ok) {  
        throw new Error("Failed");  
    }  
  
    return response.json();  
}
```

Create student

22.67 JAVASCRIPT FETCH API

ES2026

```
async function createStudent(student) {  
  
    const response =  
        await fetch("/api/students", {  
  
        method: "POST",  
  
        headers: {  
            "Content-Type": "application/json"  
        },  
  
        body: JSON.stringify(student)  
  
    });  
  
    if (!response.ok) {  
        throw new Error("Creation failed");  
    }  
  
    return response.json();  
}
```

Update student

22.67 JAVASCRIPT FETCH API

ES2026

```
async function updateStudent(id, data) {  
  
    const response =  
        await fetch(`/api/students/${id}`, {  
  
        method: "PATCH",  
  
        headers: {  
            "Content-Type": "application/json"  
        },  
  
        body: JSON.stringify(data)  
  
    });  
  
    if (!response.ok) {  
        throw new Error("Update failed");  
    }  
  
    return response.json();  
}
```

Delete student

```
async function deleteStudent(id) {  
  
    const response =  
        await fetch(`/api/students/${id}`, {  
  
            method: "DELETE"  
  
        });  
  
    if (!response.ok) {  
        throw new Error("Delete failed");  
    }  
}
```

22.68 পুরো CRUD একসাথে

```
// READ  
const students =  
    await fetch("/api/students")  
        .then(res => res.json());  
  
// CREATE  
await fetch("/api/students", {  
  
    method: "POST",  
  
    headers: {  
        "Content-Type": "application/json"  
    },  
  
    body: JSON.stringify({  
        name: "Shariar",  
        department: "CSE"  
    })  
});  
  
// UPDATE  
await fetch("/api/students/1", {  
  
    method: "PATCH",  
  
    headers: {  
        "Content-Type": "application/json"  
    },  
  
    body: JSON.stringify({  
        department: "Software Engineering"  
    })  
});  
  
// DELETE  
await fetch("/api/students/1", {  
  
    method: "DELETE"  
  
});
```

22.69 Frontend Developer হিসেবে API শেখার গুরুত্ব

React application-এ তুমি প্রায়ই করবে:

22.69 JAVASCRIPT FETCH API

ES2026

```
useEffect(() => {  
  fetch("/api/products")  
    .then(response => response.json())  
    .then(data => {  
      setProducts(data);  
    });  
}, []);
```

অথবা:

22.69 JAVASCRIPT FETCH API

ES2026

```
const response = await fetch("/api/products");  
const data = await response.json();
```

Node.js/Express backend-এর সাথে:

```
React  
↓  
Fetch/Axios  
↓  
Express API  
↓  
MongoDB/PostgreSQL
```

এটাই Full Stack development-এর core workflow-এর একটি অংশ।

MUST KNOW 🔥 22.70 Must Know — HTTP & REST API Pillars

এই Chapter থেকে অবশ্যই জানতে হবে:

HTTP

- HTTP
- Request
- Response
- URL
- Headers
- Body

HTTP Methods**OUTPUT / TEXT**

```
GET
POST
PUT
PATCH
DELETE
HEAD
OPTIONS
```

Status Codes**OUTPUT / TEXT**

```
200
201
204
400
401
403
404
409
422
429
500
503
```

REST

- REST
- Resource
- RESTful URL
- Path parameter
- Query parameter
- CRUD
- Idempotency basic concept

Fetch**JAVASCRIPT FETCH API**

ES2026

```
fetch()
```

- GET
- POST
- PUT
- PATCH
- DELETE
- Headers
- Body
- `JSON.stringify()`
- `response.json()`
- `response.text()`
- `response.ok`
- `response.status`
- `response.headers`

Advanced

- CORS
- Preflight
- `OPTIONS`
- `AbortController`

- Timeout
- Pagination
- Filtering
- Sorting
- Authentication
- Authorization
- 401 vs 403
- Network debugging

GET

JAVASCRIPT FETCH API

ES2026

```
const response = await fetch("/api/users");

const data = await response.json();
```

POST

JAVASCRIPT FETCH API

ES2026

```
await fetch("/api/users", {

  method: "POST",

  headers: {
    "Content-Type": "application/json"
  },

  body: JSON.stringify({
    name: "Shariar"
  })

});
```

PATCH

JAVASCRIPT FETCH API

ES2026

```
await fetch("/api/users/1", {

  method: "PATCH",

  headers: {
    "Content-Type": "application/json"
  },

  body: JSON.stringify({
    name: "New Name"
  })

});
```

DELETE

JAVASCRIPT FETCH API

ES2026

```
await fetch("/api/users/1", {
  method: "DELETE"
});
```

Error Check

JAVASCRIPT FETCH API

ES2026

```
if (!response.ok) {
  throw new Error(
    `HTTP ${response.status}`
  );
}
```

⚙️ Query Parameter

JAVASCRIPT FETCH API

ES2026

```
const params = new URLSearchParams({
  search: "laptop",
  page: 1
});

fetch(`/api/products?${params}`);
```

Authorization

JAVASCRIPT FETCH API

ES2026

```
fetch("/api/profile", {

  headers: {
    Authorization: `Bearer ${token}`
  }

});
```

Abort

```
const controller =  
  new AbortController();  
  
fetch(url, {  
  signal: controller.signal  
});  
  
controller.abort();
```



Practice 1 — GET

একটি function তৈরি করো:

```
JAVASCRIPT  FETCH  API
```

```
ES2026
```

```
getProducts()
```

যেটি:

```
OUTPUT / TEXT
```

```
/api/products
```

থেকে data আনবে।

Practice 2 — POST

একটি function:

```
JAVASCRIPT  FETCH  API
```

```
ES2026
```

```
createUser()
```

তৈরি করো যেটি এই data পাঠাবে:

```
JAVASCRIPT  FETCH  API
```

```
ES2026
```

```
{
  name: "Sharir",
  email: "sharir@example.com"
}
```

Practice 3 — PATCH

Product `id = 10` -এর শুধু price update করো:

```
OUTPUT / TEXT
```

```
85000
```

Practice 4 — DELETE

Product `id = 20` delete করার function তৈরি করো।

Practice 5 — Search

User input:

```
OUTPUT / TEXT
```

```
laptop
```

থেকে URL তৈরি করো:

```
OUTPUT / TEXT
```

```
/api/products?search=laptop
```

Practice 6 — Error Handling

এমন code লেখো যেখানে:

```
JAVASCRIPT  FETCH  API
```

```
ES2026
```

```
response.ok === false
```

হলে custom error throw করবে।

Practice 7 — Full CRUD

একটি:

OUTPUT / TEXT

Student Management API

এর জন্য frontend-side functions তৈরি করে:

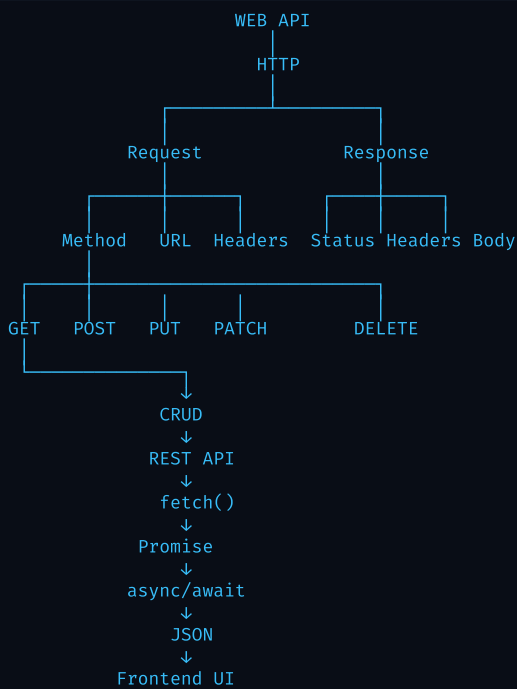
JAVASCRIPT FETCH API

ES2026

```
getStudents()
createStudent()
updateStudent()
deleteStudent()
```

SUMMARY 🗺️ Chapter 22 Final Mental Map — Web API, HTTP & Data Pipeline

পুরো Chapter-টি এইভাবে মনে রাখো:



🌟🌟 সবচেয়ে গুরুত্বপূর্ণ বাস্তব Flow:

```
User clicks button
↓
JavaScript event
↓
fetch()
↓
HTTP Request
↓
Backend API
↓
Database
↓
HTTP Response
↓
response.json()
↓
JavaScript data
↓
DOM / React State
↓
UI Update
```

Chapter 21 + 22 একসাথে করলে এখন তুমি JavaScript-এর asynchronous programming থেকে শুরু করে বাস্তব API communication পর্যন্ত একটি complete foundation তৈরি করে ফেলেছ।

NEXT CHAPTER 📖 Chapter 23 Preview — Advanced Regular Expressions (RegExp)

পরবর্তী অধ্যায়ে আমরা জাভাস্ক্রিপ্টের অন্যতম শক্তিশালী টেক্রাট প্রসেসিং ইঞ্জিন **Advanced Regular Expressions (RegExp)** নিয়ে বিস্তারিত শিখব— Lookaheads, Lookbehinds, Named Capturing Groups, Unicode Property Escapes, এবং বাস্তব ডেটা ভ্যালিডেশন প্যাটার্নস।